

# LeBrick n' Place

A vision-guided pick-and-place system that uses the UR3e robotic arm to detect, sort, and arrange LEGO bricks by colour and size into predefined designs.

41069 Robotics Studio 2 -- University of Technology Sydney

Team: **LeBrick James**

Client: Isira Wijegunawardana | Coach: Katie Powell

| Name                | Student ID | Role                      |
|---------------------|------------|---------------------------|
| Danish Silva        | 24638001   | Perception & Mapping      |
| Hari Mahadevan      | 24801045   | Interaction & Execution   |
| Benjamin Costarella | 24834727   | Motion Planning & Control |
| Dheeraj Panjwani    | 14275073   | Voice Interface & HRI     |

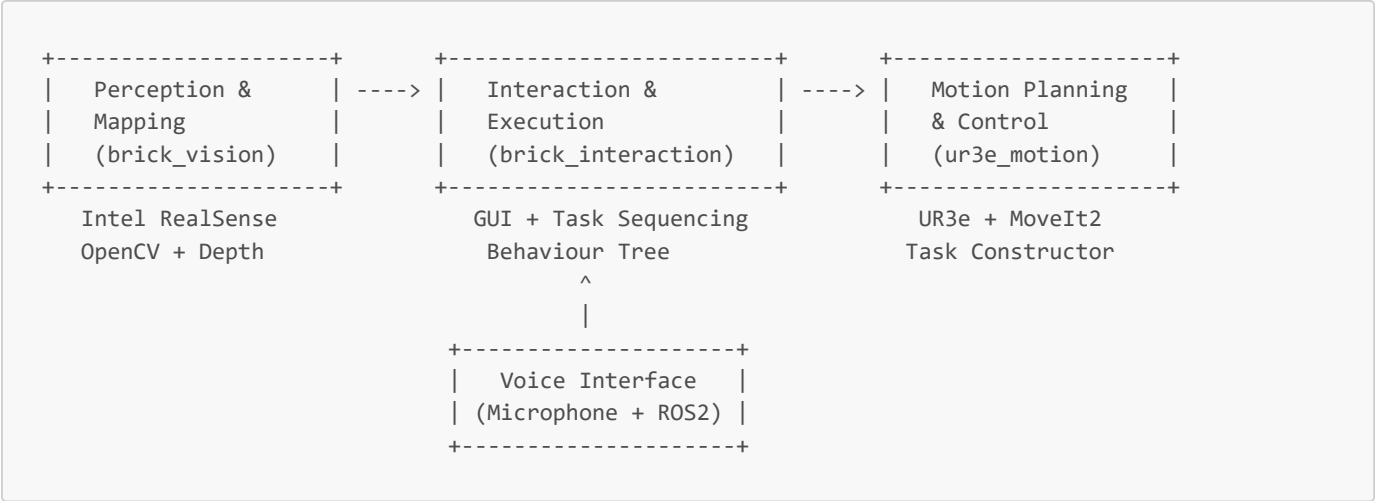
## Project Overview

LeBrick n' Place is a ROS2-based vision-guided pick-and-place system that uses the UR3e robotic arm to detect, sort, and arrange custom LEGO bricks onto the placement board. Utilising an overhead Intel RealSense D435i depth camera, the system identifies the bricks colour, size, and pose, allowing the UR3e to pick-and-place them in the users desired arrangement. Users interact through a GUI that supports colour sorting, size sorting, horizontal/vertical placement, and preset build designs.

### Subsystems

| Subsystem                 | Package                                | Lead                | Description                                                                                                                               |
|---------------------------|----------------------------------------|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Perception & Mapping      | brick_vision                           | Danish Silva        | Brick detection, colour/size classification, and 3D pose estimation via RealSense + OpenCV                                                |
| Motion Planning & Control | ur3e_motion_mtc                        | Benjamin Costarella | Performs Inverse Kinematics, trajectory planning, singularity & obstacle avoidance using MoveIt2 to plan and execute pick-and-place tasks |
| Interaction & Execution   | brick_interaction & brick_gui          | Hari Mahadevan      | Task sequencing, state management, GUI coordination, and error recovery                                                                   |
| Voice Interface           | Task Level Control via Voice Interface | Dheeraj Panjwani    | Voice commands via microphone for system control and task selection                                                                       |

### System Architecture



Repository Structure

```
BrickPickNPlace-RS2/
├── src/
│   ├── brick_vision/           # Perception & mapping (RealSense + OpenCV)
│   ├── brick_gui/             # User interface
│   ├── brick_interaction/      # Task sequencing & execution logic
│   ├── ur3e_motion_mtc/        # Motion planning (MoveIt2 Task Constructor)
│   └── voice_interface/        # Voice & keyboard command interface
│       ├── voice_interface/
│       │   ├── voice_input_node.py
│       │   ├── command_parser_node.py
│       │   ├── system_command_listener.py
│       │   └── reset_executor_node.py
│       └── package.xml
├── .gitignore
└── README.md
```

Dependencies

Hardware

| Component   | Details                             |
|-------------|-------------------------------------|
| Robot       | Universal Robots UR3e               |
| Gripper     | OnRobot RG2                         |
| Camera      | Intel RealSense D435i               |
| LEGO Bricks | LeBrick n' Place Custom LEGO Bricks |
| LEGO Board  | LeBrick n' Place Custom LEGO Board  |

Software

| Component        | Version          |
|------------------|------------------|
| Operating System | Ubuntu 22.04     |
| ROS2             | Humble Hawksbill |
| MoveIt2          | 2.5.9            |

| Component               | Version |
|-------------------------|---------|
| Movelt Task Constructor | 0.1.3   |
| RViz2                   | 11.2.26 |
| Python                  | 3.10+   |
| C++                     | 17      |

## Installation

### Hardware Setup

| Component  | Details                                                  |
|------------|----------------------------------------------------------|
| Robot      | UR3e with gripper attachment                             |
| Camera     | Intel RealSense D435i, mounted overhead on a fixed stand |
| Workspace  | White LEGO base plate                                    |
| Bricks     | Large LEGO bricks (4x2 = 100mm x 50mm, 15mm studs)       |
| Microphone | Computer headset or USB microphone                       |

### Software

#### 1. Install build tools, ROS2 packages, and Movelt Task Constructor:

```
sudo apt update
sudo apt install -y \
  python3-colcon-common-extensions \
  python3-rosdep \
  ros-humble-moveit \
  ros-humble-ur-robot-driver \
  ros-humble-cv-bridge \
  ros-humble-moveit-task-constructor-*
```

#### 2. Install Python packages:

```
pip install opencv-python pyrealsense2 numpy SpeechRecognition pyaudio
```

If `pip install pyaudio` fails, install the system dependency first:

```
sudo apt install portaudio19-dev
pip install pyaudio
```

#### 3. Initialise rosdep:

```
sudo rosdep init
rosdep update
```

#### 4. Clone the repository:

```
git clone https://github.com/DanishUTS/BrickPickNPlace-RS2.git
cd BrickPickNPlace-RS2
```

## 5. Install workspace dependencies:

```
rosdep install --from-paths src --ignore-src -r -y
```

## 6. Build the workspace:

```
source /opt/ros/humble/setup.bash
colcon build
source install/setup.bash
```

---

# Launch Commands

### Terminal 1 — UR driver (connect to physical robot):

```
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch ur_onrobot_control start_robot.launch.py \
  ur_type:=ur3e \
  onrobot_type:=rg2 \
  robot_ip:=0.0.0.0 \
  launch_rviz:=false
```

### Terminal 1 — UR driver (connect to Fake Hardware):

```
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch ur_onrobot_control start_robot.launch.py ur_type:=ur3e onrobot_type:=rg2
use_fake_hardware:=true launch_rviz:=false
```

### Terminal 2 — MoveIt + RViz:

```
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch ur_onrobot_moveit_config ur_onrobot_moveit.launch.py ur_type:=ur3e onrobot_type:=rg2
launch_rviz:=true
```

### Terminal 3 — Brick Detection:

```
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 run brick_vision brick_detector
```

### Terminal 4 — Motion Planning and Control:

```
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch ur3e_motion_mtc ur3e_motion_mtc.launch.py
```

**Terminal 5 — Brick Interaction:**

```
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 run brick_interaction brick_interaction_node
```

**Terminal 6 — Voice Interface: Voice Input Node:**

```
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 run voice_interface voice_input_node
```

**Terminal 7 — Voice Interface: Command Parser Node:**

```
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 run voice_interface command_parser_node
```

**Terminal 8 — Voice Interface: System Command Listener:**

```
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 run voice_interface system_command_listener
```

**Terminal 9 — Voice Interface: Reset Executor Node:**

```
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 run voice_interface reset_executor_node
```

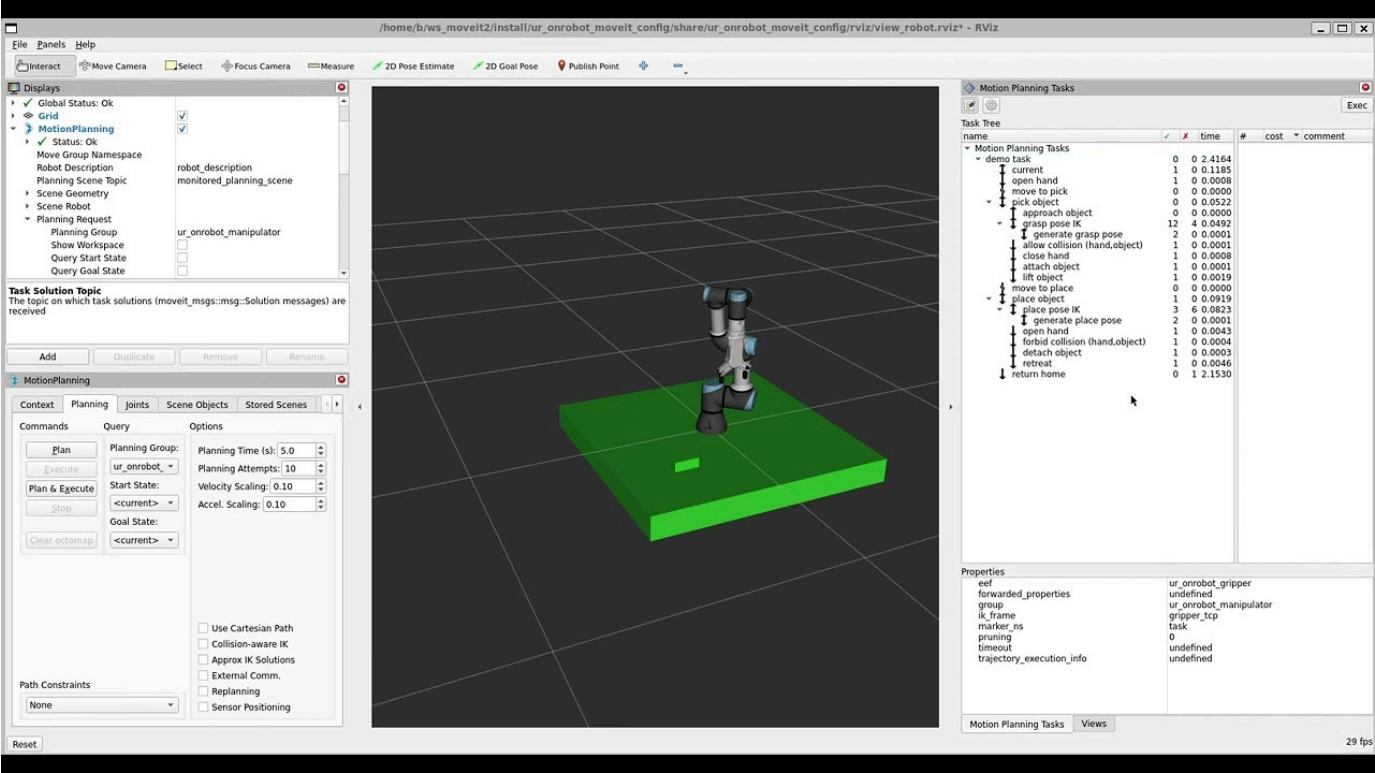
**Terminal 10 — GUI:**

```
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 run brick_gui brick_gui_node
```

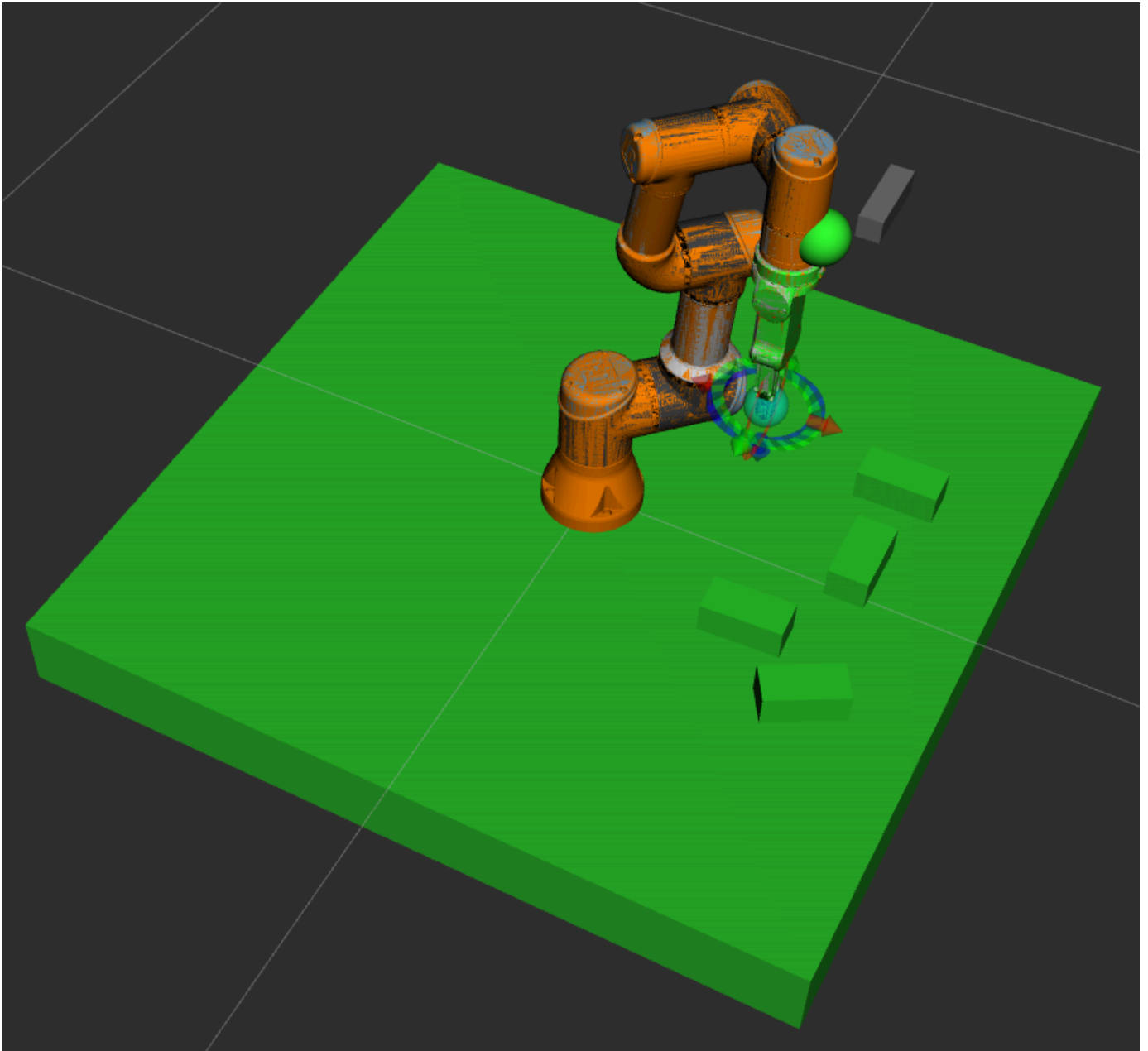
---

## Demo and System Images

### Demonstration Video

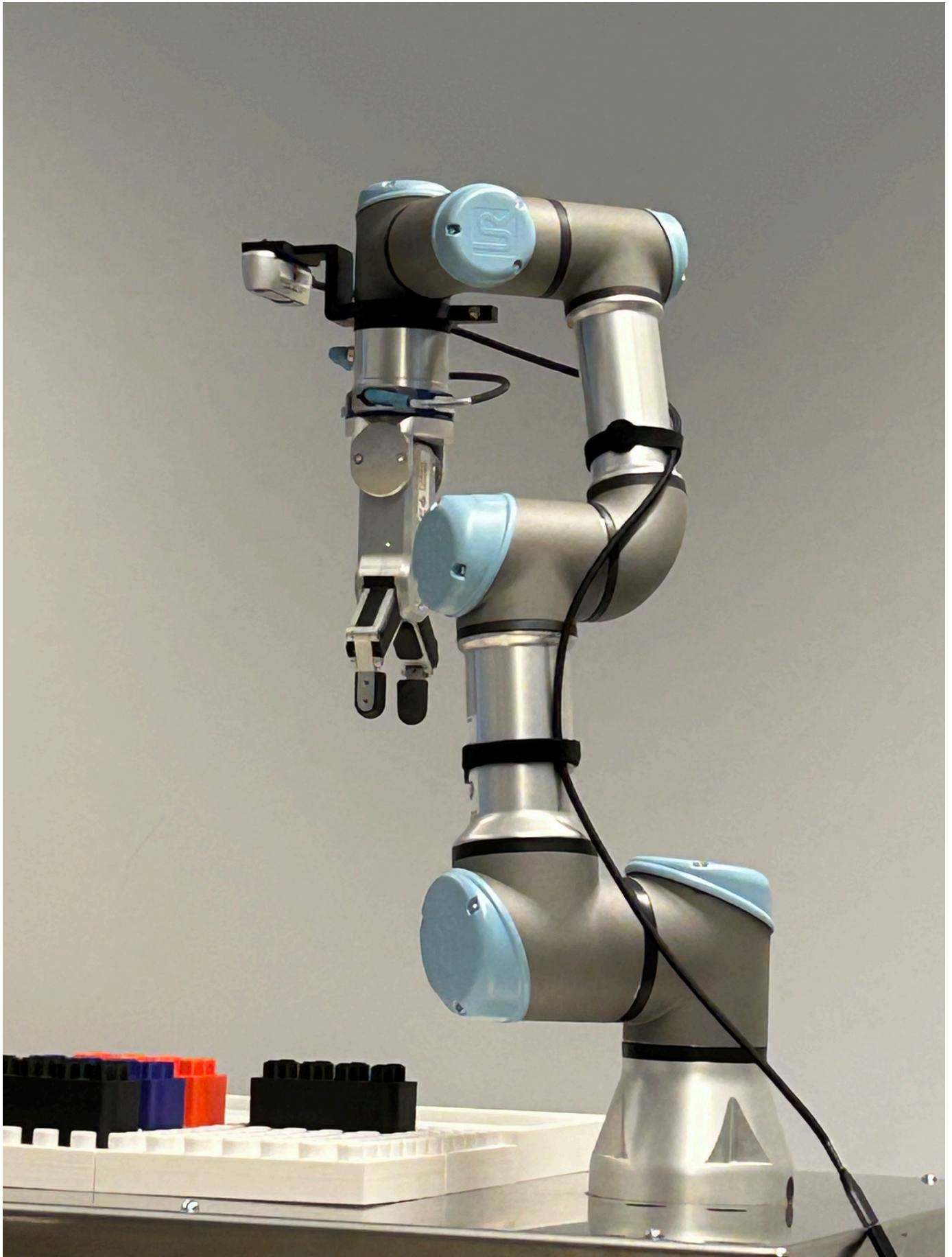


System Integration



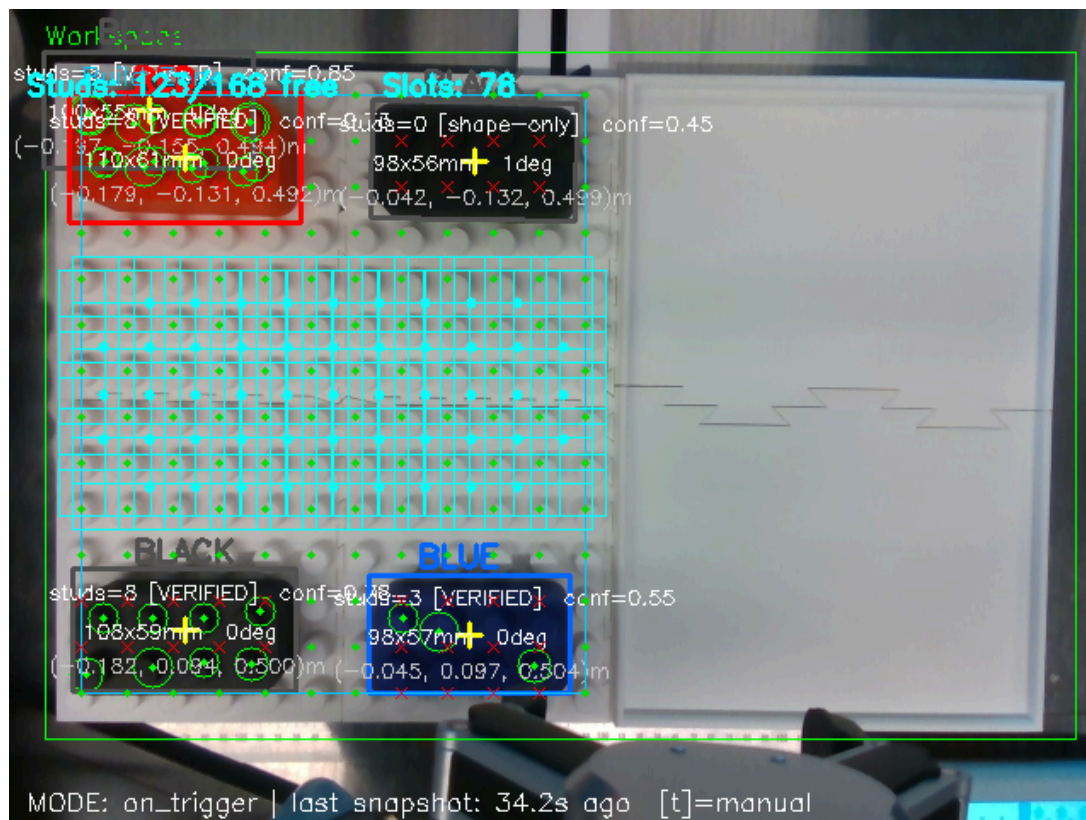
Robot Setup





Camera Vision





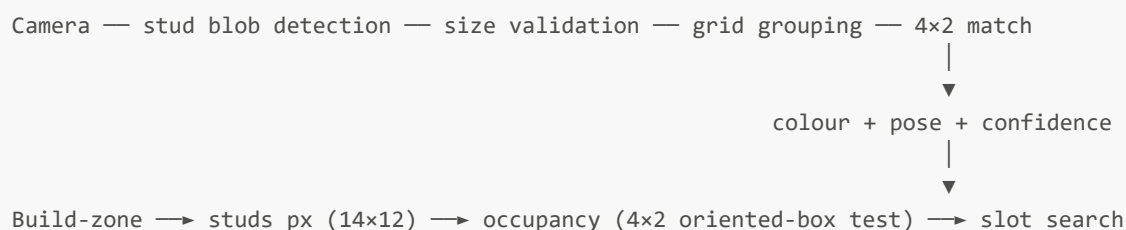
## brick\_vision — Perception & Mapping

ROS 2 package for detecting LEGO bricks and analysing a build plate using an **Intel RealSense D435i** depth camera and **OpenCV**. Identifies bricks by their stud pattern, classifies colour, estimates 3D pose, and finds available placement slots on a 12×14-stud build zone.

### What the node does

For each captured frame the pipeline runs three stages:

1. **Brick detection** — find 4×2 LEGO bricks in the workspace ROI, identify their colour, compute centre + yaw in 3D camera coordinates.
2. **Build-zone analysis** — locate the 14-row × 12-column stud grid on the build plate, decide which studs are free vs occupied, slide a 4×2 window over the grid to find legal placement slots (with a 1-stud gap rule).
3. **Publish results** — bricks, free studs, and available slots are all published on dedicated topics for the interaction node to consume.



Two operating modes

| Mode                              | When to use                                                                                                                                                                                                                                                                                             | Frame rate   |
|-----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| <code>on_trigger</code> (default) | Production. Vision waits for an <code>std_msgs/Empty</code> on <code>/snapshot_trigger</code> (e.g. when the arm reaches home and the workspace is clear), takes 5 frames, keeps the best one, publishes once. Results are <b>latched</b> so late subscribers immediately get the most recent snapshot. | Event-driven |
| <code>continuous</code>           | Debug / tuning. Runs detection at ~15 Hz the whole time.                                                                                                                                                                                                                                                | ~15 Hz       |

## Two-stage calibration

Both calibrations are interactive (clicks in the OpenCV window) and saved to `~/.brick_vision/workspace_calibration.json`.

| Calibration            | Trigger            | What you click                                                                                                                                                      |
|------------------------|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Workspace ROI</b>   | <code>c</code> key | Drag a rectangle around the table area the camera should look at. Surface depth is sampled inside the ROI.                                                          |
| <b>Build-zone grid</b> | <code>b</code> key | Click the <i>top-left</i> stud centre, then the <i>bottom-right</i> stud centre, of the 12×14 build plate. The 168-stud grid is linearly interpolated between them. |

## ROS 2 interface

### Parameters

| Parameter                    | Default                          | Description                                                      |
|------------------------------|----------------------------------|------------------------------------------------------------------|
| <code>mode</code>            | <code>"on_trigger"</code>        | <code>"on_trigger"</code> or <code>"continuous"</code>           |
| <code>trigger_topic</code>   | <code>"/snapshot_trigger"</code> | Topic name to subscribe to in <code>on_trigger</code> mode       |
| <code>snapshot_frames</code> | 5                                | Capture N frames per trigger, publish the best one               |
| <code>show_preview</code>    | <code>true</code>                | Show the OpenCV window. Set <code>false</code> for headless runs |

### Subscribed topics

| Topic                          | Type                        | Description                                                                                                                         |
|--------------------------------|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>/snapshot_trigger</code> | <code>std_msgs/Empty</code> | ( <code>on_trigger</code> mode only) Take one snapshot now. Topic name is overridable via the <code>trigger_topic</code> parameter. |

### Published topics

All published under the node namespace `/brick_detector/`. In `on_trigger` mode the four result topics use **TRANSIENT\_LOCAL** QoS (latched, depth 1) — a subscriber that starts after a snapshot immediately receives the most recent result.

| Topic                          | Type                                      | Purpose                                                                                                                                                                                                                                                                                   |
|--------------------------------|-------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>~/detection_image</code> | <code>sensor_msgs/Image</code>            | Annotated camera frame: brick boxes + colour labels, build-zone perimeter, free studs (green dots), occupied studs (red X), available 4×2 slots (cyan rectangles), status line. Not latched.                                                                                              |
| <code>~/detections</code>      | <code>vision_msgs/Detection3DArray</code> | <b>All</b> detected bricks (pickup + placed). Useful for rviz / debugging. One entry per brick, binding <code>pose + size + colour + confidence</code> . Pose orientation encodes yaw as a quaternion (roll/pitch = 0). Colour string is in <code>results[0].hypothesis.class_id</code> . |

| Topic                            | Type                                      | Purpose                                                                                                                                                                                                                                                                                                                     |
|----------------------------------|-------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>~/pickup_detections</code> | <code>vision_msgs/Detection3DArray</code> | <b>Main consumer topic for the interaction node.</b> Only bricks whose pixel centre falls <i>outside</i> the calibrated build-zone rectangle — i.e. bricks waiting to be picked. Same message structure as <code>~/detections</code> . If the build zone isn't calibrated, this is identical to <code>~/detections</code> . |
| <code>~/placed_detections</code> | <code>vision_msgs/Detection3DArray</code> | Bricks already on the build plate (pixel centre inside the build-zone rectangle, ½-stud margin). Diagnostic only — the interaction node should ignore these.                                                                                                                                                                |
| <code>~/brick_pose</code>        | <code>geometry_msgs/PoseStamped</code>    | Convenience — first <b>pickup-side</b> brick only. Use <code>~/pickup_detections</code> for real consumption.                                                                                                                                                                                                               |
| <code>~/free_studs</code>        | <code>geometry_msgs/PoseArray</code>      | Centre of every free stud on the 12×14 grid. Diagnostic.                                                                                                                                                                                                                                                                    |
| <code>~/available_slots</code>   | <code>geometry_msgs/PoseArray</code>      | <b>Placement-target topic.</b> Midpoint of each legal 4×2 area (free + 1-stud gap respected). Orientation encodes the slot's yaw (long axis along X vs Y).                                                                                                                                                                  |

All headers stamp `frame_id = "camera_color_optical_frame"` — consumers should transform to `base_link` via `tf2`.

## Brick detection details

### Pipeline

1. **Capture** — RealSense streams aligned colour + depth at 640×480 @ 30 fps.
2. **Workspace ROI** — Crop to the calibrated rectangle (border pixels excluded).
3. **HSV pre-segmentation** — Build colour masks for the supported colour set.
4. **Contour extraction + size check** — Keep contours whose oriented bounding box has the right aspect ratio ( $2:1 \pm 35\%$ ) and metric size ( $\sim 100 \times 50 \text{ mm} \pm 30\%$ ).
5. **Stud verification (HoughCircles)** — Inside each candidate, find circular studs and check there are roughly 8 of them at the expected spacing.
6. **Pose + yaw** — Centre from contour moments, yaw from the oriented box, Z from median depth in a small window.
7. **Confidence** — `0.65 max` from colour+size, plus `0.35 bonus` if  $\geq 2$  verified studs were found (total  $\leq 1.0$ ).

### Supported colours

| Colour | HSV ranges         |
|--------|--------------------|
| Red    | hue 0–10 + 170–179 |
| Orange | hue 11–25          |
| Yellow | hue 26–34          |
| Green  | hue 35–85          |
| Blue   | hue 86–130         |
| Purple | hue 131–169        |
| Black  | V < 60, S < 120    |

White is intentionally excluded — the workspace surface is white.

### Per-brick output dictionary (internal)

```
{
  "center_px":    (320, 240),          # pixel coordinates
  "center_3d":    [0.05, -0.02, 0.35], # metres, camera frame
  "size_px":      (80, 40),
  "size_m":       (0.098, 0.049),
  "angle":        45.0,                # degrees, used as yaw
  "colour":       "orange",
  "colour_bgr":   (0, 140, 255),
  "stud_count":   8,
  "brick_config": (4, 2),
  "confidence":   0.92,
}
```

These get repackaged into `vision_msgs/Detection3DArray` when published.

## Build-zone analysis

### Geometry

- **Grid:** 12 columns × 14 rows of studs (168 studs total).
- **Stud pitch:** 25 mm centre-to-centre (matches LEGO spec).
- **Brick footprint:** 4×2 studs (the only size currently used).
- **Placement gap rule:** every placed brick needs **1 row + 1 column of free studs around it** before another brick can be placed.

### How occupancy is decided

For each detected brick the node builds an oriented bounding box on the image, then checks every stud's pixel position against that box (using a rotated-coords inside-test). Studs falling inside any brick's footprint are marked occupied; everything else is free.

### How slots are found

A sliding window scans both orientations (4×2 and 2×4) across the grid. A slot is *available* iff:

- All 8 studs inside the window are free.
- Every stud in the surrounding 1-stud ring is either free or outside the grid (the gap-ring check).

The midpoint of each accepted slot, plus its orientation (yaw 0 or  $\pi/2$ ), is published in `~/available_slots`.

### `analyze_buildzone()` return value

```
{
  "studs_px":      np.ndarray,   # (14, 12, 2) pixel positions
  "studs_3d":      np.ndarray,   # (14, 12, 3) metres, camera frame
  "occupancy":     np.ndarray,   # (14, 12) bool
  "free_studs":    [ {"row":r, "col":c, "px":(x,y), "xyz":(x,y,z)}, ... ],
  "slots":         [ {"row":r, "col":c, "long_axis":"x"|"y",
                      "yaw":0.0| $\pi/2$ , "xyz":(x,y,z)}, ... ],
}
```

## Usage

### Build

```
cd ~/git/BrickPickNPlace-RS2
colcon build --packages-select brick_vision
source install/setup.bash
```

Run (on\_trigger mode, default)

```
ros2 run brick_vision brick_detector
```

First run prompts for workspace + build-zone calibration via the OpenCV window — see keyboard controls below. Calibration is then loaded automatically on every subsequent run.

Trigger a snapshot

Three ways:

| Method                          | Command                                                                                         |
|---------------------------------|-------------------------------------------------------------------------------------------------|
| Manual key                      | Press <b>t</b> in the preview window                                                            |
| ROS topic                       | <code>ros2 topic pub --once /snapshot_trigger std_msgs/msg/Empty "{}"</code>                    |
| Programmatic (interaction node) | Publish <code>std_msgs/Empty</code> on <code>/snapshot_trigger</code> when the arm reaches home |

Continuous mode (debug)

```
ros2 run brick_vision brick_detector --ros-args -p mode:=continuous
```

Headless

```
ros2 run brick_vision brick_detector --ros-args -p show_preview:=false
```

Standalone (no ROS, OpenCV GUI only)

```
python3 src/brick_vision/brick_vision/brick_detector.py
```

Launch file

```
ros2 launch brick_vision brick_detection.launch.py
```

Keyboard controls (preview window)

| Key | Action                                                                      |
|-----|-----------------------------------------------------------------------------|
| q   | Quit                                                                        |
| t   | <b>Manual trigger</b> — works in both modes, useful for independent testing |
| c   | Re-calibrate workspace ROI (drag rectangle)                                 |

| Key      | Action                                                |
|----------|-------------------------------------------------------|
| <b>b</b> | Re-calibrate build zone (click TL stud, then BR stud) |
| <b>s</b> | Save the current annotated frame as a PNG             |

## Configuration methods (what you can change, and how)

Three layers, from most static to most dynamic:

| Layer                          | Where                                                                      | When to use                                                                                                                             |
|--------------------------------|----------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| <b>Compile-time constants</b>  | Top of <code>brick_detector.py</code> (e.g. <code>STUD_DIAMETER_M</code> ) | Physical facts about the brick/scene and detector thresholds. Rebuild after change.                                                     |
| <b>ROS 2 parameters</b>        | <code>--ros-args -p name:=value</code>                                     | Runtime behaviour: mode, trigger topic, preview, frames-per-snap. No rebuild.                                                           |
| <b>Interactive calibration</b> | OpenCV window keys <b>c</b> and <b>b</b>                                   | Geometry that depends on the <i>physical setup</i> — where the workspace is in the image and where the build plate sits. Saved to disk. |

### Calibration file

Saved canonically at `~/.brick_vision/workspace_calibration.json`. Contents:

```
{
  "roi": [x, y, w, h],           // workspace rectangle (pixels)
  "surface_depth": 0.42,        // metres, table top
  "build_zone_tl": [px, py],    // top-left stud centre (pixels)
  "build_zone_br": [px, py]     // bottom-right stud centre (pixels)
}
```

Override path with `BRICK_VISION_CALIBRATION` env var. A legacy fallback at `src/brick_vision/config/workspace_calibration.json` is read if the canonical file is missing — kept so existing dev setups keep working.

## Parameter reference (what each one *means* and *when to tune*)

Physical brick / stud geometry (rebuild required)

| Constant                               | Default | What it controls                              | When to change                                         |
|----------------------------------------|---------|-----------------------------------------------|--------------------------------------------------------|
| <code>BRICK_LENGTH_M</code>            | 0.100   | 4-stud axis length, used in metric-size check | Different brick size, e.g. swapping to 6×2             |
| <code>BRICK_WIDTH_M</code>             | 0.050   | 2-stud axis length                            | Same — must match <code>BRICK_LENGTH_M</code> ratio    |
| <code>STUD_DIAMETER_M</code>           | 0.015   | Expected stud radius in HoughCircles          | Different stud type (DUPLO $\approx$ 0.03 m)           |
| <code>STUD_PITCH_M</code>              | 0.025   | Stud centre-to-centre spacing                 | LEGO is 8 mm pitch — leave it. Only changes for DUPLO. |
| <code>BRICK_STUDS_LONG × _SHORT</code> | 4 × 2   | Slot-search window size                       | Match the brick footprint you're picking               |

Detection tolerances (most-tuned parameters)

| Constant                        | Default             | Effect of increasing                  | Effect of decreasing                          | Symptom-driven tuning                                                                        |
|---------------------------------|---------------------|---------------------------------------|-----------------------------------------------|----------------------------------------------------------------------------------------------|
| <b>ASPECT_TOL</b>               | 0.35                | More candidates accepted (incl. junk) | Stricter — may reject viewed-at-angle bricks  | If bricks are missed when seen from oblique angles, raise to ~0.45                           |
| <b>SIZE_TOL</b>                 | 0.30                | Tolerates depth-noise size error      | Rejects bricks where depth is wrong           | Raise if depth is noisy / surface_depth is off                                               |
| <b>MIN_CONTOUR_AREA</b>         | 500 px <sup>2</sup> | Rejects more noise blobs              | Catches small bricks far from camera          | Lower if the camera is mounted far away; raise if false positives appear in clutter          |
| <b>HOUGH_RADIUS_TOL</b>         | 0.50                | More circles found                    | Stricter — stud verification fails more often | Raise (e.g. 0.6) if you see bricks rejected for "studs missing" but the colour/size is right |
| <b>MIN_CIRCULARITY</b> (legacy) | 0.55                | Stricter circle shape                 | Tolerates elliptical studs (camera at angle)  | Lower when the camera looks down at >30°                                                     |

### Confidence formula (how scores are built)

```
confidence = base + bonus, clamped to [0, 1]
base = CONF_BASE_MAX - aspect_err - size_err # 0.65 max
bonus = (studs_found / 8) × CONF_STUD_BONUS if studs_found ≥ 2 else 0 # 0.35 max
```

| Constant                   | Default | Meaning                                                                                |
|----------------------------|---------|----------------------------------------------------------------------------------------|
| <b>CONF_BASE_MAX</b>       | 0.65    | Cap on shape-only confidence. Forces stud verification to push above 0.65.             |
| <b>CONF_STUD_BONUS</b>     | 0.35    | How much weight stud verification adds. Higher = harder to be confident without studs. |
| <b>MIN_STUDS_FOR_BONUS</b> | 2       | Threshold — fewer studs and the brick gets no bonus, even if its shape is perfect.     |

### Colour thresholds

```
COLOUR_RANGES = [
    ("red", [(0,90,90),(10,255,255)], [(170,90,90),(179,255,255)]),
    ("orange", [(11,90,90),(25,255,255)]),
    ...
]
BLACK_V_MAX = 60 # value (brightness) ceiling for black detection
BLACK_S_MAX = 120 # saturation ceiling for black
```

| Tuning need                                         | Change                                                          |
|-----------------------------------------------------|-----------------------------------------------------------------|
| Red bricks misclassified as orange under warm light | Move the red/orange boundary up: (11,...) → (15,...)            |
| Black bricks missed under bright lights             | Raise <b>BLACK_V_MAX</b> to 80                                  |
| Black detected on coloured bricks in shadow         | Lower <b>BLACK_V_MAX</b> to 40, and/or lower <b>BLACK_S_MAX</b> |

### Build-zone constants

| Constant                                        | Default | Description                                                      |
|-------------------------------------------------|---------|------------------------------------------------------------------|
| <b>BUILD_GRID_COLS</b> × <b>BUILD_GRID_ROWS</b> | 12 × 14 | Stud-grid size on the plate. Change for a different build plate. |



| Constant                         | Default | Description                                                             |
|----------------------------------|---------|-------------------------------------------------------------------------|
| <code>PLACEMENT_GAP_STUDS</code> | 1       | Required free-stud ring around each placed brick. 0 = bricks can touch. |

## ROS 2 parameters

| Parameter                    | Default                          | Tuning rationale                                                                                                                                                             |
|------------------------------|----------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>mode</code>            | <code>"on_trigger"</code>        | Use <code>"continuous"</code> only for debug — it wastes CPU and produces jittery poses while the arm is mid-motion.                                                         |
| <code>trigger_topic</code>   | <code>"/snapshot_trigger"</code> | Rename if another node owns this name — keep <code>std_msgs/Empty</code> .                                                                                                   |
| <code>snapshot_frames</code> | 5                                | Raise (8–10) for noisier scenes, lower (3) for faster cycles. More frames = more chance the best one is clean, at the cost of trigger-to-publish latency (~30 ms per frame). |
| <code>show_preview</code>    | <code>true</code>                | <code>false</code> for headless/SSH runs and when running on the robot PC.                                                                                                   |

## Troubleshooting

### Detection problems

| Symptom                                                    | Likely cause                                             | Fix                                                                                                                                                                                  |
|------------------------------------------------------------|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>No bricks detected, ever</b>                            | Workspace ROI is wrong or missing                        | Press <code>c</code> , redraw the rectangle. Check <code>~/brick_vision/workspace_calibration.json</code> exists.                                                                    |
| <b>No bricks, but ROI is set</b>                           | Surface depth wrong → metric size check fails everything | Re-run <code>c</code> calibration; make sure the table is visible inside the rectangle.                                                                                              |
| <b>Bricks detected far from camera but missed up close</b> | Studs too big for <code>HOUGH_MIN_DIST_M</code>          | Lower <code>HOUGH_MIN_DIST_M</code> , or expect that bricks need to stay within ~30–50 cm.                                                                                           |
| <b>Bricks detected up close but missed far away</b>        | Studs become sub-pixel                                   | Move the camera closer, lower the resolution requirement ( <code>MIN_CONTOUR_AREA</code> ).                                                                                          |
| <b>One colour consistently missed</b>                      | Colour range needs tuning under the actual lights        | Open the OpenCV "Colour Mask" debug window (if enabled) or sample HSV with <code>cv2.cvtColor</code> on a saved snapshot. Adjust the corresponding <code>COLOUR_RANGES</code> entry. |
| <b>Random false positives in the background</b>            | ROI too large                                            | Press <code>c</code> , draw a tighter rectangle.                                                                                                                                     |
| <b>Confidence stuck at 0.6–0.65 (never higher)</b>         | Studs aren't being verified                              | Stud verification is failing — usually <code>HOUGH_RADIUS_TOL</code> is too tight or camera angle is too oblique. Raise tolerance or move the camera more vertical.                  |

### Build-zone problems

| Symptom                                                  | Likely cause                                                                         | Fix                                                                                       |
|----------------------------------------------------------|--------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| <b>No slots published / no <code>~/free_studs</code></b> | Build zone not calibrated → <code>analyze_buildzone</code> returns <code>None</code> | Press <code>b</code> , click TL then BR stud of the build plate.                          |
| <b>Studs/grid drawn but at wrong angle</b>               | The 2-click calibration assumes axis-aligned grid                                    | Either physically align the plate with the camera axes, or upgrade to 4-click perspective |

| Symptom                                                            | Likely cause                                                 | Fix                                                                                             |
|--------------------------------------------------------------------|--------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
|                                                                    |                                                              | calibration                                                                                     |
| <b>A brick is detected on the plate but its studs show as free</b> | Brick yaw is far off from what the oriented-box test expects | Check that <code>det["angle"]</code> is sane; verify oriented bounding box dims match the brick |
| <b>Adjacent bricks treated as one slot</b>                         | <code>PLACEMENT_GAP_STUDS = 0</code>                         | Set back to 1                                                                                   |

## ROS-side problems

| Symptom                                                   | Likely cause                                                                                            | Fix                                                                                                                                                              |
|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Trigger published, nothing happens</b>                 | Mode is <code>continuous</code> (trigger ignored) or topic mismatch                                     | Check <code>ros2 param get /brick_detector mode</code> ; check <code>ros2 topic info /snapshot_trigger -v</code> shows your publisher and the node's subscriber. |
| <b>Late-starting interaction node sees no messages</b>    | <code>mode=continuous</code> (volatile QoS) — switch to <code>on_trigger</code> for latched delivery    | Or change QoS in <code>brick_detector.py:1040</code> for continuous mode too                                                                                     |
| <b><code>detection_image</code> won't display in rviz</b> | QoS mismatch — rviz default Image QoS is volatile, the topic is volatile, but reliability may not match | Set rviz's Image display reliability to "Best Effort" or check <code>ros2 topic info -v</code>                                                                   |
| <b>Snapshot trigger fires but no detections</b>           | Workspace is occluded mid-snapshot (e.g. arm in frame)                                                  | Trigger from <code>home</code> pose only; raise <code>snapshot_frames</code> so best-of-N is more likely clean                                                   |

## Camera problems

| Symptom                                                    | Likely cause                                               | Fix                                                                                                                                    |
|------------------------------------------------------------|------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| <b><code>get_frames()</code> returns <code>None</code></b> | RealSense not connected / claimed by another process       | <code>pkill -f realsense; lsusb   grep Intel</code> ; check <code>realsense-viewer</code> works                                        |
| <b>Depth values are 0 everywhere</b>                       | Camera too close (< 0.10 m) or surface is dark/transparent | Move camera to 0.3–0.7 m; ensure the build plate is matte and lit                                                                      |
| <b>Big depth jitter on the same stud</b>                   | Frame-to-frame noise on D435i                              | This is why we average inside <code>_sample_depth</code> (median of a 21×21 sample window). Increase the sample window if still noisy. |

## Reference docs

A full subsystem deep-dive — architecture, every parameter explained with tuning rationale, viva-style Q&A, troubleshooting flowcharts — is available at:

```
docs/brick_vision_subsystem.docx
```

Regenerate with:

```
python3 tools/generate_docs.py
```

## Files

```
brick_vision/
├── brick_vision/
│   ├── __init__.py
│   ├── brick_detector.py           # main detector + ROS node
│   ├── brick_detector_no_buildzone.py # backup: state before build-zone work
│   └── brick_detector_dimension_based.py # legacy contour-only detector
├── config/
│   └── workspace_calibration.json    # legacy fallback (canonical is ~/.brick_vision/)
├── launch/
│   └── brick_detection.launch.py
├── package.xml
└── setup.py
```

## How the interaction node consumes this

Brick poses are published in the **camera frame** (`camera_color_optical_frame`). The interaction node is responsible for transforming them to `base_link` via `tf2` — see `brick_interaction/pose_transform_test.py` for a working example.

Typical pick flow:

1. Arm reaches home → interaction node publishes `Empty` to `/snapshot_trigger`.
2. Vision captures 5 frames, picks the best, publishes latched results on `~/detections` and `~/available_slots`.
3. Interaction node reads `~/detections`, picks a brick (e.g. by colour), transforms its pose to `base_link`, sends the goal to MoveIt.
4. After place, arm returns to home → repeat.

The latched QoS guarantees the interaction node sees the most recent snapshot even if it starts up after the vision node.

## Brick Interaction Subsystem

### Purpose

The `brick_interaction` package is the interaction and execution coordinator for the Brick Pick and Place system. It connects the GUI, vision, motion, and gripper subsystems into one controlled pick-and-place workflow.

The GUI is provided by the separate `brick_gui` package, but it is part of the same interaction/control section. The GUI publishes user commands to `/brick_command`, and `brick_interaction` receives those commands, runs the state machine, and publishes status updates back to the GUI on `/system_status`.

The subsystem is responsible for:

- Receiving user commands such as `start`, `pause`, `stop`, `gripper_open`, and `gripper_close`
- Managing the system state with a deterministic state machine
- Requesting fresh brick and slot data from the vision subsystem
- Selecting the nearest eligible brick and a suitable available placement slot
- Sending pick/place targets to the motion subsystem
- Sending gripper open/close commands to the RG2 gripper controller
- Publishing system status for the GUI

This package does not perform camera detection or MoveIt planning itself. Instead, it coordinates those subsystems through ROS 2 topics.

### Key Topics, Services, Files, and Nodes

#### Main Nodes

```
brick_interaction
brick_gui
```

Run the interaction node directly:

```
ros2 run brick_interaction brick_interaction_node
```

Run the GUI directly:

```
ros2 run brick_gui brick_gui_node
```

Main Files

| File                                     | Purpose                                                                                                                                          |
|------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| brick_interaction/interaction_node.py    | Main ROS 2 node. Handles commands, scan requests, pick/place selection, motion commands, gripper commands, and status publishing.                |
| brick_interaction/state_machine.py       | Pure Python state machine for <code>idle</code> , <code>running</code> , <code>paused</code> , <code>completed</code> , and <code>error</code> . |
| brick_interaction/brick_sorter.py        | Brick and placement-slot data models, detection conversion, confidence/colour/size filtering, and nearest-brick/slot selection.                  |
| brick_interaction/motion_client.py       | Publishes pick/place pose arrays to the motion subsystem on <code>/ordered_pose_array</code> .                                                   |
| brick_interaction/gripper_client.py      | Publishes smooth RG2 finger-width commands to <code>/finger_width_controller/commands</code> .                                                   |
| launch/brick_interaction.launch.py       | Launches the main <code>brick_interaction</code> node.                                                                                           |
| ../brick_gui/brick_gui/gui_node.py       | PyQt5 GUI node. Publishes Start/Pause/Stop commands and displays system status.                                                                  |
| ../brick_gui/brick_gui/ui/main_window.ui | Qt Designer layout for the GUI buttons, status badge, task label, and log window.                                                                |
| ../brick_gui/setup.py                    | Installs the GUI node and <code>.ui</code> file so <code>ros2 run brick_gui brick_gui_node</code> can load the interface.                        |

Services

This subsystem currently does not provide or call ROS services. Communication is topic-based.

Inputs and Outputs

Subscribed Inputs

| Topic                       | Type                         | Source                    | Purpose                                                                                                                             |
|-----------------------------|------------------------------|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| <code>/brick_command</code> | <code>std_msgs/String</code> | GUI or manual CLI command | Receives <code>start</code> , <code>pause</code> , <code>stop</code> , <code>gripper_open</code> , and <code>gripper_close</code> . |

| Topic                                        | Type                                      | Source           | Purpose                                                                                             |
|----------------------------------------------|-------------------------------------------|------------------|-----------------------------------------------------------------------------------------------------|
| <code>/brick_detector/detections</code>      | <code>vision_msgs/Detection3DArray</code> | Vision subsystem | Provides detected bricks with pose, size, colour/class, and confidence.                             |
| <code>/brick_detector/available_slots</code> | <code>geometry_msgs/PoseArray</code>      | Vision subsystem | Provides open placement/drop-zone slots.                                                            |
| <code>/brick_detector/brick_pose</code>      | <code>geometry_msgs/PoseStamped</code>    | Vision subsystem | Provides a fallback or best single brick pose. This topic does not include colour or size metadata. |

## Published Outputs

| Topic                                          | Type                                    | Destination             | Purpose                                                                                                                                    |
|------------------------------------------------|-----------------------------------------|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <code>/snapshot_trigger</code>                 | <code>std_msgs/Empty</code>             | Vision subsystem        | Requests a fresh camera snapshot/scan.                                                                                                     |
| <code>/ordered_pose_array</code>               | <code>std_msgs/Float64MultiArray</code> | Motion subsystem        | Sends pick and place target poses.                                                                                                         |
| <code>/system_status</code>                    | <code>std_msgs/String</code>            | GUI or monitoring tools | Publishes current state: <code>idle</code> , <code>running</code> , <code>paused</code> , <code>completed</code> , or <code>error</code> . |
| <code>/finger_width_controller/commands</code> | <code>std_msgs/Float64MultiArray</code> | RG2 gripper controller  | Sends desired gripper finger width in metres.                                                                                              |

## GUI Interface

The GUI uses the same ROS interfaces as the command-line tests:

| GUI Control  | Published Topic             | Message                                                    |
|--------------|-----------------------------|------------------------------------------------------------|
| Start button | <code>/brick_command</code> | <code>std_msgs/String</code> with data: <code>start</code> |
| Pause button | <code>/brick_command</code> | <code>std_msgs/String</code> with data: <code>pause</code> |
| Stop button  | <code>/brick_command</code> | <code>std_msgs/String</code> with data: <code>stop</code>  |

The GUI subscribes to `/system_status` and updates:

- Status badge: `idle`, `running`, `paused`, `completed`, or `error`
- Current task label
- Last update time
- Execution log

The current GUI does not expose `gripper_open` or `gripper_close` buttons. Those manual gripper commands can still be tested from the command line.

## Motion Output Format

In the current `mtc` mode, `/ordered_pose_array` contains exactly 12 values:

```
[brick_x, brick_y, brick_z, brick_roll, brick_pitch, brick_yaw,
 slot_x, slot_y, slot_z, slot_roll, slot_pitch, slot_yaw]
```

This is the contract expected by the `ur3e_motion_mtc` subsystem.

## How to Run or Test Independently

Before running commands, build and source the workspace:

```
colcon build --packages-select brick_interaction brick_gui
source install/setup.bash
```

Important: run each `ros2 topic echo` in its own separate terminal before publishing the command you want to observe. Most command and motion topics are not latched, so if you start echoing after a one-shot message has already been published, you may miss it.

### Terminal 1: Start the Interaction Node

```
ros2 run brick_interaction brick_interaction_node
```

Expected behaviour:

- The node starts as `brick_interaction`
- It publishes initial status `idle`
- It waits for GUI or CLI commands

### Terminal 2: Observe Scan Requests

Start this before sending `start`:

```
ros2 topic echo /snapshot_trigger
```

When the system starts, this terminal should print an empty message. This proves the interaction node is publishing scan requests to the vision subsystem.

### Terminal 3: Observe Motion Commands

Start this before sending `start`:

```
ros2 topic echo /ordered_pose_array
```

When the node receives fresh detections and slots, this terminal should print the selected pick/place pair. This proves the interaction node is publishing to the motion subsystem.

### Terminal 4: Observe Gripper Commands

Start this before testing manual gripper commands:

```
ros2 topic echo /finger_width_controller/commands
```

This proves the interaction node is publishing gripper width commands.

### Terminal 5: Start the GUI

```
ros2 run brick_gui brick_gui_node
```

Expected behaviour:

- The GUI window opens.
- The Start, Pause, and Stop buttons are visible.
- The status badge shows the latest system state.
- Clicking Start/Pause/Stop sends the matching command to the interaction node.

Simple GUI command check:

- Click **Start** and confirm the GUI status changes to **RUNNING** and Terminal 1 logs the **running** state.
- Click **Pause** while the system is running and confirm the GUI status changes to **PAUSED** and Terminal 1 logs the **paused** state.
- Click **Stop** and confirm the GUI status returns to **IDLE** and Terminal 1 logs the **idle** state.

## Terminal 6: Start Continuous Mock Available Slots

Use a continuous publisher instead of `--once` so the interaction node receives a fresh slot message after it enters the **running** state.

```
ros2 topic pub /brick_detector/available_slots geometry_msgs/msg/PoseArray "{poses: [{position: {x: 0.20, y: 0.40, z: 0.0}, orientation: {w: 1.0}}, {position: {x: 0.60, y: 0.40, z: 0.0}, orientation: {w: 1.0}}]}" --rate 1
```

This provides two open placement slots.

## Terminal 7: Start Continuous Mock Brick Detections

This example publishes two bricks. The blue brick is closer to **ROBOT\_BASE = (0.0, 0.0)** than the red brick, so the interaction node should select the blue brick first.

```
ros2 topic pub /brick_detector/detections vision_msgs/msg/Detection3DArray "{detections: [{bbox: {center: {position: {x: 0.55, y: 0.25, z: 0.0}, orientation: {w: 1.0}}, size: {x: 0.032, y: 0.016, z: 0.012}}, results: [{hypothesis: {class_id: red, score: 0.90}}]}, {bbox: {center: {position: {x: 0.25, y: 0.05, z: 0.0}, orientation: {w: 1.0}}, size: {x: 0.032, y: 0.016, z: 0.012}}, results: [{hypothesis: {class_id: blue, score: 0.95}}]}]}" --rate 1
```

This proves the interaction node is subscribed to `/brick_detector/detections` and can process multiple candidate bricks.

The closest-brick check works because `brick_sorter.py` compares each brick's XY position to **ROBOT\_BASE = (0.0, 0.0)**:

| Brick | Position     | Distance from arm base                     | Expected order |
|-------|--------------|--------------------------------------------|----------------|
| Blue  | (0.25, 0.05) | $\sqrt{0.25^2 + 0.05^2} = 0.255 \text{ m}$ | 1st            |
| Red   | (0.55, 0.25) | $\sqrt{0.55^2 + 0.25^2} = 0.604 \text{ m}$ | 2nd            |

So the node should select the blue brick first. In Terminal 1, the node logs should show the blue brick above the red brick in the sorted summary. In Terminal 3, `/ordered_pose_array` should start with the blue brick pose:

```
data:
- 0.25
- 0.05
- 0.0
```

```
- 0.0
- 0.0
- 0.0
...
```

The first six values are the selected brick pose: **x**, **y**, **z**, **roll**, **pitch**, and **yaw**. Since the blue brick is closer, the first two values should be approximately **0.25** and **0.05**, not **0.55** and **0.25**.

## Start the Cycle With the GUI

Once the observer terminals and mock vision publishers are running, click the **Start** button in the GUI.

Expected behaviour:

- Terminal 1 logs that the interaction node entered **running**.
- Terminal 2 receives a **/snapshot\_trigger**.
- Terminal 3 receives an **/ordered\_pose\_array** motion command.
- The GUI status badge changes to **RUNNING**.

## Alternative: Start the Cycle Without the GUI

If you do not want to use the GUI, publish the same command manually:

```
ros2 topic pub --once /brick_command std_msgs/msg/String "{data: start}"
```

Expected behaviour:

- Terminal 1 logs that the interaction node entered **running**
- The node logs that it requested a fresh snapshot
- The node logs that it received available slots
- The node logs that it received brick detections
- The node prints a sorted brick summary
- The closest brick should appear first in the sorted summary
- The node logs the selected pick/place pair
- **/ordered\_pose\_array** receives 12 values

Expected proof in the terminals:

| Terminal                               | What it proves                                               |
|----------------------------------------|--------------------------------------------------------------|
| Terminal 2, <b>/snapshot_trigger</b>   | The node requests a fresh scan from vision.                  |
| Terminal 3, <b>/ordered_pose_array</b> | The node publishes a selected brick/slot pair to motion.     |
| Terminal 5, GUI                        | The GUI sends commands and displays status.                  |
| Terminal 6, slots publisher            | The node can subscribe to available placement slots.         |
| Terminal 7, detections publisher       | The node can subscribe to brick detections.                  |
| Terminal 1, node logs                  | The node sorted bricks and selected the nearest valid brick. |

After proving the motion command, send **stop** so the loop does not keep requesting scans:

```
ros2 topic pub --once /brick_command std_msgs/msg/String "{data: stop}"
```

Or click the **Stop** button in the GUI.



### Timeout Test

To prove failure handling, stop the mock vision publishers, then click Start in the GUI or send:

```
ros2 topic pub --once /brick_command std_msgs/msg/String "{data: start}"
```

If the vision topics do not respond, the node waits for `SCAN_TIMEOUT_S` seconds and then enters the `error` state.

Expected result: the GUI status changes to `ERROR`, and Terminal 1 logs the scan timeout.

This proves the subsystem can detect missing vision input and fail safely instead of continuing with stale data.

### Manual Gripper Test

Manual gripper commands are only accepted when the pick-and-place cycle is not running.

Start observing `/finger_width_controller/commands` in a separate terminal before sending these commands.

Close:

```
ros2 topic pub --once /brick_command std_msgs/msg/String "{data: gripper_close}"
```

Open:

```
ros2 topic pub --once /brick_command std_msgs/msg/String "{data: gripper_open}"
```

Expected behaviour:

- `gripper_open` publishes finger widths moving toward `GRIPPER_OPEN_WIDTH`
- `gripper_close` publishes finger widths moving toward `GRIPPER_CLOSE_WIDTH`
- Manual gripper commands do not advance the pick-and-place cycle

### Configurable Settings and Important Parameters

The current implementation uses constants in `interaction_node.py`. These are the main values to tune:

| Setting                                    | Default            | Meaning                                                                                         |
|--------------------------------------------|--------------------|-------------------------------------------------------------------------------------------------|
| <code>Z_APPROACH</code>                    | <code>0.15</code>  | Safe clearance height above a brick or slot in metres.                                          |
| <code>Z_GRAB</code>                        | <code>0.005</code> | Grab/release height above the surface in metres.                                                |
| <code>GRIPPER_OPEN_WIDTH</code>            | <code>0.110</code> | RG2 fully open finger width in metres.                                                          |
| <code>GRIPPER_CLOSE_WIDTH</code>           | <code>0.0</code>   | RG2 closed finger width in metres.                                                              |
| <code>GRIPPER_SPEED</code>                 | <code>0.05</code>  | Gripper finger travel speed in metres per second.                                               |
| <code>GRIPPER_UPDATE_HZ</code>             | <code>20.0</code>  | Rate for publishing intermediate gripper widths.                                                |
| <code>MOTION_MODE</code>                   | <code>'mtc'</code> | Uses the MTC pick/place interface. Legacy <code>'stepwise'</code> mode also exists in the code. |
| <code>MTC_MOTION_COMPLETION_DELAY_S</code> | <code>45.0</code>  | Time to wait before assuming the MTC pick/place task is complete.                               |
| <code>SCAN_TIMEOUT_S</code>                | <code>8.0</code>   | Maximum time to wait for fresh vision detections and slots.                                     |
| <code>MAX_BRICKS_PER_RUN</code>            | <code>100</code>   | Safety limit on bricks placed in one run.                                                       |

| Setting                  | Default | Meaning                                               |
|--------------------------|---------|-------------------------------------------------------|
| ELIGIBLE_COLOURS         | []      | Optional colour allow-list. Empty means any colour.   |
| ELIGIBLE_SIZES           | []      | Optional brick-size allow-list. Empty means any size. |
| MIN_DETECTION_CONFIDENCE | 0.30    | Minimum accepted detection confidence.                |

`brick_sorter.py` also contains fallback/mock values:

| Setting         | Purpose                                                             |
|-----------------|---------------------------------------------------------------------|
| ROBOT_BASE      | XY reference point used for sorting bricks by distance.             |
| MOCK_BRICKS     | Representative mock brick list for testing without perception data. |
| PLACEMENT_SLOTS | Static fallback placement slots if no live slot data is available.  |

Future improvement: expose these values through ROS parameters or a YAML configuration file so they can be tuned without editing source code.

## Expected Behaviour

When a `start` command is received:

1. The state machine transitions to `running`.
2. The node publishes `/snapshot_trigger`.
3. The node waits for fresh `/brick_detector/detections` and `/brick_detector/available_slots`.
4. Detections are converted into `Brick` objects.
5. Slots are converted into `PlacementSlot` objects.
6. Bricks are filtered by confidence, colour, and size if those filters are configured.
7. The nearest eligible brick is selected.
8. The nearest available slot is selected.
9. A pick/place pair is published on `/ordered_pose_array`.
10. After the configured MTC completion delay, the node requests another scan.
11. If no bricks remain, the state becomes `completed`.

Command behaviour:

| Current State         | Command                                                | Result                                    |
|-----------------------|--------------------------------------------------------|-------------------------------------------|
| idle                  | start                                                  | Begin scan and pick/place cycle.          |
| running               | pause                                                  | Transition to <code>paused</code> .       |
| running               | stop                                                   | Return to <code>idle</code> .             |
| paused                | start                                                  | Resume by entering <code>running</code> . |
| paused                | stop                                                   | Return to <code>idle</code> .             |
| completed             | start                                                  | Start a new cycle.                        |
| error                 | start                                                  | Retry by starting a new cycle.            |
| Any non-running state | <code>gripper_open</code> / <code>gripper_close</code> | Manually jog gripper.                     |

## Known Limitations and Assumptions

- The subsystem assumes that the vision coordinates are suitable for motion. Comments in the code note that a TF transform from `camera_color_optical_frame` to the robot base frame may be required once calibration is finalized.
- Motion completion is currently timer-based. `MotionClient` assumes the MTC task has completed after `MTC_MOTION_COMPLETION_DELAY_S`; it does not currently receive action feedback from the motion subsystem.

- Most configurable values are source-code constants rather than ROS parameters.
- `/brick_detector/brick_pose` is only a fallback/best-pose topic and does not include brick colour or size.
- In current `mtc` mode, the system sends one brick/slot pair per scan and then requests a new scan after the assumed motion completion.
- The node requires both fresh detections and fresh available slots after a scan trigger. If either is missing, it times out and enters `error`.
- The `stepwise` motion mode is present as legacy logic, but the current integrated mode is `mtc`.
- Static `MOCK_BRICKS` and `PLACEMENT_SLOTS` are useful for early testing, but final operation should use calibrated vision outputs.
- Manual gripper commands are ignored while the pick-and-place cycle is running to avoid disrupting the automated sequence.
- The GUI Pause button changes the interaction state and cancels scan waiting, but it does not cancel or pause an already-sent motion task because the current motion interface is publisher/timer-based rather than action-feedback-based.

Troubleshooting

| Symptom                                                    | Likely Cause                                                                         | What to Check                                                                                                                      |
|------------------------------------------------------------|--------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| Status becomes <code>error</code> after <code>start</code> | Vision did not publish both detections and available slots before timeout.           | Echo <code>/brick_detector/detections</code> , <code>/brick_detector/available_slots</code> , and <code>/snapshot_trigger</code> . |
| No motion command appears                                  | Missing detections, missing slots, or all detections filtered out.                   | Echo <code>/ordered_pose_array</code> and check node logs for filtering messages.                                                  |
| Gripper does not move                                      | Gripper controller may not be running or topic name may not match.                   | Echo <code>/finger_width_controller/commands</code> .                                                                              |
| Manual gripper command ignored                             | System is currently <code>running</code> .                                           | Send <code>stop</code> first, then send <code>gripper_open</code> or <code>gripper_close</code> .                                  |
| Pick/place target appears wrong                            | Input brick or slot coordinates may not match the motion subsystem's expected frame. | Check the values published on <code>/brick_detector/detections</code> and <code>/brick_detector/available_slots</code> .           |

UR3e\_Motion\_MTC — Motion Planning & Control Subsystem

Purpose

The Motion Planning and Control is responsible for all the physical movements of the UR3e robotic arm. This subsystem receives a brick's position within the pickup zone, and the target placement position on the LEGO board. Utilising MoveIt Task Constructor, the UR3e plans and executes the full pick-and-place sequence while avoiding collisions with itself and the surrounding environment.

System Requirements

| Component               | Version                               |
|-------------------------|---------------------------------------|
| Operating System        | Ubuntu 22.04 (via WSL2 on Windows 11) |
| ROS2                    | Humble Hawksbill                      |
| MoveIt2                 | 2.5.9                                 |
| MoveIt Task Constructor | 0.1.3                                 |
| RViz2                   | 11.2.26                               |

| Component   | Version               |
|-------------|-----------------------|
| WSL Version | WSL2 version 2.5.10.0 |
| Robot       | Universal Robots UR3e |
| Gripper     | OnRobot RG2           |

## Key Nodes, Topics, and Files

### Node

| Node                     | Description                                                                                                                                                                           |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>motion_node</code> | This ROS2 Node waits for pose data to be recieved from <code>ordered_pose_array</code> , then sets up the planning scene, planning and executing the pick-and-place task on the UR3e. |

### Subscribed Topics

| Topic                           | Type                                        | Description                                                                                                                                                                        |
|---------------------------------|---------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>ordered_pose_array</code> | <code>std_msgs/msg/Float64MultiArray</code> | get placement position. See Inputs section for format.<br>Consists of 12 float values representing the brick pickup and target placement positions. See Inputs section for format. |

### Key Source Files

| File                                          | Description                                                                                                                                                                                       |
|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>src/ur3e_motion_mtc.cpp</code>          | This is the main source file, containing the <code>MTCTaskNode</code> class, environment setup, pose subscriber, and <code>main()</code> .                                                        |
| <code>launch/ur3e_motion_mtc.launch.py</code> | This is the Launch File, loading the robot description, kinematics parameters, and planning configurations from <code>ur_onrobot_moveit_config</code> , and starts the <code>motion_node</code> . |

### External Dependencies

| Package                                                 | Purpose                                                    |
|---------------------------------------------------------|------------------------------------------------------------|
| <code>rcpp</code>                                       | ROS2 client library for nodes, publishers, and subscribers |
| <code>moveit_core</code>                                | Contains core Moveit funtionality                          |
| <code>moveit_task_constructor_core</code>               | Contains core MTC planning pipeline                        |
| <code>std_msgs</code>                                   | <code>Float64MultiArray</code> message type for pose input |
| <code>tf2_geometry_msgs</code> , <code>tf2_eigen</code> | Quaternion and transform utilities                         |

## Inputs and Outputs

### Input — `ordered_pose_array`

A single `std_msgs/msg/Float64MultiArray` message containing 12 float64 values representing two poses in order:

```
[brick_x, brick_y, brick_z, brick_roll, brick_pitch, brick_yaw, target_x, target_y, target_z, target_roll, target_pitch, target_yaw]
```

| Index | Value        | Unit    |
|-------|--------------|---------|
| 0     | brick x      | metres  |
| 1     | brick y      | metres  |
| 2     | brick z      | metres  |
| 3     | brick roll   | radians |
| 4     | brick pitch  | radians |
| 5     | brick yaw    | radians |
| 6     | target x     | metres  |
| 7     | target y     | metres  |
| 8     | target z     | metres  |
| 9     | target roll  | radians |
| 10    | target pitch | radians |
| 11    | target yaw   | radians |

All positions must be in the `world` frame, which within the current setup is equivalent to the `base_link`. The makes the coordinates relative to the base of the UR3e.

The node waits for the the `ordered_pose_array` to be received before planning and executing movements on the UR3e.

Output — Robot Motion

On receiving valid pose data the node executes this sequence on the UR3e:

```
1. Open gripper
2. Move arm to above-brick position           (sampling planner)
3. Descend toward brick                       (Cartesian planner, downward -Z)
4. Close gripper around brick
5. Attach brick to gripper in scene
6. Lift brick upward                         (Cartesian planner, upward +Z)
7. Move arm to above-target position          (sampling planner)
8. Lower to place position                   (Cartesian planner)
9. Open gripper
10. Detach brick from gripper in scene
11. Retreat upward                          (Cartesian planner, upward +Z)
12. Return arm to camera_home configuration (sampling planner)
```

How to Run

Building

```
cd ~/ws_moveit2
colcon build --symlink-install --packages-select ur3e_motion_mtc
source install/setup.bash
```

Launch Commands

Terminal 1 — UR driver (connect to physical robot):

```
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch ur_onrobot_control start_robot.launch.py \
  ur_type:=ur3e \
  onrobot_type:=rg2 \
  robot_ip:=0.0.0.0 \
  launch_rviz:=false
```

#### Terminal 1 — UR driver (connect to Fake Hardware):

```
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch ur_onrobot_control start_robot.launch.py ur_type:=ur3e onrobot_type:=rg2
use_fake_hardware:=true launch_rviz:=false
```

#### Terminal 2 — MoveIt + RViz:

```
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch ur_onrobot_moveit_config ur_onrobot_moveit.launch.py ur_type:=ur3e onrobot_type:=rg2
launch_rviz:=true
```

#### Terminal 3 — Launch Motion Planning and Control Node:

```
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch ur3e_motion_mtc ur3e_motion_mtc.launch.py
```

#### Terminal 4 — Test Pose Publish:

```
ros2 topic pub --once /ordered_pose_array std_msgs/msg/Float64MultiArray "{
  data: [0.3, 0.3, 0.02, 0.0, 0.0, 0.0,
        -0.25, 0.2, 0.03, 0.0, 0.0, 0.0]
}"
```

### Troubleshooting

If the MTC motion planning completes, but the execution fails, this may be an issue with the controllers.

#### Controller List:

First check which controllers are active and inactive.

```
ros2 control list_controllers
```

#### Activate Gripper Trajectory Controller:

```
ros2 control switch_controllers \
  --activate finger_width_trajectory_controller \
```

```
--deactivate finger_width_controller
```

### Activate Scaled Joint Trajectory Controller:

```
ros2 control switch_controllers \
  --activate scaled_joint_trajectory_controller \
  --deactivate joint_trajectory_controller
```

### Opening MoveIt Task Constructor in RViz

On the bottom left window, click the **Add** button.

Under `moveit_task_constructor_visualization`, select **Motion Planning Task**.

In **Display** under **Motion Planning Task**, change **Task Solution Topic** to `/solution`.

## Configurable Parameters

All parameters are set directly in `src/ur3e_motion_mtc.cpp`:

| Parameter                          | Location in code                     | Default                   | Description                                             |
|------------------------------------|--------------------------------------|---------------------------|---------------------------------------------------------|
| Approach min/max distance          | <code>approach object</code> stage   | <code>0.0 / 0.20 m</code> | Vertical descent range before grasping                  |
| Lift min/max distance              | <code>lift object</code> stage       | <code>0.0 / 0.20 m</code> | Vertical lift range after grasping                      |
| Retreat min/max distance           | <code>retreat</code> stage           | <code>0.0 / 0.20 m</code> | Vertical retreat range after placing                    |
| <code>move to pick</code> timeout  | <code>stage_move_to_pick</code>      | <code>15.0 s</code>       | Max planning time to reach above the brick              |
| <code>move to place</code> timeout | <code>stage_move_to_place</code>     | <code>15.0 s</code>       | Max planning time to reach above the place position     |
| Max IK solutions (grasp)           | <code>ComputeIK</code> grasp wrapper | <code>16</code>           | IK solutions explored for grasp pose                    |
| Max IK solutions (place)           | <code>ComputeIK</code> place wrapper | <code>8</code>            | IK solutions explored for place pose                    |
| <code>task_.plan()</code> count    | <code>doTask()</code>                | <code>10</code>           | Number of full task solutions to find before executing  |
| Velocity scaling                   | <code>cartesian_planner</code>       | <code>0.5</code>          | Fraction of max joint velocity for Cartesian stages     |
| Acceleration scaling               | <code>cartesian_planner</code>       | <code>0.5</code>          | Fraction of max joint acceleration for Cartesian stages |

## Known Limitations and Assumptions

### Assumptions:

- The pose data received from `ordered_pose_array` is in the `world` frame of reference.
- The pose data received from `ordered_pose_array` is an array of 12 float values representing the brick's pose and target pose, 6 vales for each.
- The node will only receive a single pick-and-place task per motion, and will only receive a new one once returning to the `camera_home` robot configuration.
- Two brick's will be distances far enough appart that the gripper can pick and place each brick using the brick's long axis.

- All required controllers have been activated before launching the `motion_node`.

**Known Limitations:**

- If planning fails, the node logs an error and exits. It does not retry automatically with different parameters.
- The workspace does not model the environment of the actually UR3e. There is no placement board or other bricks, so the robot doesn't avoid collision with the environment yet.
- Currently, the Cartesian movements only work if the minimum is set to 0m, so they dont retreat or approach the bricks.

## Voice Interface Subsystem

The Voice Interface subsystem provides task-level control of the robot through voice commands via microphone or keyboard input. It translates spoken or typed commands into ROS2 messages published to the shared `/brick_command` topic, which is consumed by the Interaction & Execution subsystem.

### Nodes

| Node                                 | File                                    | Description                                                                                                                                   |
|--------------------------------------|-----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>voice_input_node</code>        | <code>voice_input_node.py</code>        | Captures microphone or keyboard input and publishes raw text to <code>/voice_command_raw</code>                                               |
| <code>command_parser_node</code>     | <code>command_parser_node.py</code>     | Subscribes to <code>/voice_command_raw</code> , normalises and maps input, publishes parsed commands to <code>/brick_command</code>           |
| <code>system_command_listener</code> | <code>system_command_listener.py</code> | Monitors <code>/brick_command</code> and related topics — used for integration debugging and status logging                                   |
| <code>reset_executor_node</code>     | <code>reset_executor_node.py</code>     | Temporary execution node — listens to <code>/brick_command</code> and directly executes joint trajectories on the UR3e for standalone testing |

### Topics

| Topic                           | Type                                | Publisher                        | Subscriber(s)                                                                                                  | Description                                     |
|---------------------------------|-------------------------------------|----------------------------------|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------|
| <code>/voice_command_raw</code> | <code>std_msgs/String</code>        | <code>voice_input_node</code>    | <code>command_parser_node</code>                                                                               | Raw unprocessed voice or keyboard input         |
| <code>/brick_command</code>     | <code>std_msgs/String</code>        | <code>command_parser_node</code> | <code>brick_interaction</code> ,<br><code>reset_executor_node</code> ,<br><code>system_command_listener</code> | Shared command bus used by all subsystems       |
| <code>/build_request</code>     | <code>std_msgs/String</code>        | <code>command_parser_node</code> | <code>brick_interaction</code>                                                                                 | Preset build commands (BUILD_TOWER, BUILD_LINE) |
| <code>/block_sequence</code>    | <code>std_msgs/String</code>        | <code>command_parser_node</code> | <code>brick_interaction</code>                                                                                 | Custom brick placement sequence commands        |
| <code>/joint_states</code>      | <code>sensor_msgs/JointState</code> | UR driver                        | <code>reset_executor_node</code>                                                                               | Current robot joint positions                   |



| Topic          | Type            | Publisher         | Subscriber(s)           | Description                                      |
|----------------|-----------------|-------------------|-------------------------|--------------------------------------------------|
|                |                 |                   |                         | used for trajectory planning                     |
| /system_status | std_msgs/String | brick_interaction | system_command_listener | System status feedback (monitored for debugging) |

## Supported Commands

| Voice / Keyboard Input | Aliases                            | Published To    | Published Value | Behaviour                                         |
|------------------------|------------------------------------|-----------------|-----------------|---------------------------------------------------|
| start                  | begin, go                          | /brick_command  | start           | Executes demo trajectory sequence                 |
| pause                  | hold, wait                         | /brick_command  | pause           | Cancels active trajectory                         |
| stop                   | halt, cancel                       | /brick_command  | stop            | Cancels trajectory and returns robot to home pose |
| reset                  | home, go home                      | /brick_command  | reset           | Returns robot to home pose                        |
| build tower            | tower                              | /build_request  | BUILD_TOWER     | Triggers tower build preset                       |
| build line             | line                               | /build_request  | BUILD_LINE      | Triggers line build preset                        |
| sequence 1 2 3         | Spoken e.g. sequence one two three | /block_sequence | 1,2,3           | Custom brick placement sequence                   |

## Input Modes

| Mode                  | Key | Description                                                      |
|-----------------------|-----|------------------------------------------------------------------|
| Single microphone     | m   | Captures one spoken command, then returns to mode selection      |
| Continuous microphone | c   | Keeps listening and processing commands until Ctrl+C             |
| Keyboard              | k   | Accepts typed commands — useful for testing without a microphone |

## Configuration

| Parameter         | Default                                             | File                   | Description                                          |
|-------------------|-----------------------------------------------------|------------------------|------------------------------------------------------|
| device_index      | 10                                                  | voice_input_node.py    | Microphone device index — change if mic not detected |
| energy_threshold  | 100                                                 | voice_input_node.py    | Mic sensitivity — lower value = more sensitive       |
| pause_threshold   | 0.8                                                 | voice_input_node.py    | Seconds of silence before a command is finalised     |
| phrase_time_limit | 4s                                                  | voice_input_node.py    | Maximum duration of a single spoken command          |
| Home joints       | [0.8713, -1.4801, 0.1733, -0.2580, -1.5837, 5.5503] | reset_executor_node.py | Robot home pose in radians (6 joints)                |

## Voice Interface

### Microphone not detected / `device_index` error

Run the following to list available audio devices and find your mic's index:

```
python3 -c "import speech_recognition as sr; print(sr.Microphone.list_microphone_names())"
```

Update `device_index` in `voice_input_node.py` to match your microphone.

---

### Command not recognised by Google Speech Recognition

The system uses Google STT which requires an active internet connection. Check your network is available and no firewall is blocking outbound requests. Use keyboard mode `[k]` to bypass the mic and test the rest of the pipeline.

---

### Speech recognised but command not matched

The node will log: `Speech recognized, but not matched to a valid command.`

Speak one of the supported commands clearly: `start`, `pause`, `stop`, `reset`, `build tower`, `build line`. The node also accepts common aliases (e.g. `go`, `halt`, `home`) — see the Supported Commands table above.

---

### `reset_executor_node` — trajectory action server not available

The UR driver (Terminal 1) must be running before launching `reset_executor_node`. Check Terminal 1 for errors. If testing without a physical robot, ensure fake hardware is enabled:

```
use_fake_hardware:=true
```

---

### No joint states received

`reset_executor_node` requires `/joint_states` from the UR driver to plan trajectories. Confirm the driver launched successfully with:

```
ros2 topic echo /joint_states
```

---

### Commands not reaching `brick_interaction`

Verify each step in the pipeline individually:

```
# Check voice input is publishing
ros2 topic echo /voice_command_raw

# Check parser is forwarding
ros2 topic echo /brick_command

# Check build commands
ros2 topic echo /build_request
```

---

### `pyaudio` install fails on Ubuntu

Install the system dependency first:

```
sudo apt install portaudio19-dev  
pip install pyaudio
```